

Wishlist Storefront — Project Documentation

Ambiguities Discovered

The original prompt intentionally omitted several implementation details. Rather than proceeding with implicit assumptions, each ambiguity was surfaced and resolved before writing code.

#	Ambiguity	Decision Made	Reasoning
1	Is this single-user or multi-user?	Single-user, one browser profile	No auth requirement specified; adding a login system would be out of scope.
2	Where is state persisted?	localStorage	No backend specified. LocalStorage is native, requires no configuration, and survives page refreshes.
3	What defines a "duplicate" product in merge?	Same <code>product_id</code>	Product ID is the stable identity. Name or price could change; ID cannot.
4	Is merge destructive (source deleted) or additive (source kept)?	Destructive — source is deleted	"Merge" semantically means combine, not copy. Keeping the source leaves the user with redundant lists to manage manually. Full reasoning in § Why Destructive Merge .
5	What happens to duplicate items during merge?	Skip silently (set-union)	Throwing an error on duplicate would block the entire merge. Silently skipping and reporting a count is the least surprising behavior.
6	What is the product data schema?	Static seed — id, name, description, price, image_url, category	No product API or admin specified; static data keeps the project self-contained and deterministic.
7	Should wishlists have unique names?	No enforcement	The prompt did not mention it. Uniqueness is enforced only by ID. Adding name uniqueness would be a future improvement.
8	Is there a product limit per wishlist?	No hard limit	No constraint specified. Performance degrades gracefully; a very large wishlist is a future optimization concern.
9	Should the merge wizard pre-select a source?	Yes, based on which card triggered it	Reduces clicks for the common case (merging the list you are already viewing).

#	Ambiguity	Decision Made	Reasoning
10	What persistence format should be used?	JSON under <code>wishlist_store_v1</code>	Single key avoids partial-read bugs. Versioned key allows a future migration path.

Problem Statement

Build a client-side e-commerce storefront with wishlist management. The primary requirement is a reliable, atomic **wishlist merge** feature: combine two user-created wishlists into one while automatically detecting and skipping duplicate products.

The application runs entirely in the browser with no backend and no authentication. State persists in `localStorage` so wishlists survive page refreshes.

Design Evolution

This section documents how key decisions evolved during development. Showing iteration is more honest than presenting the final design as if it were obvious from the start.

Data Model — Nested → Normalized

First instinct: Store wishlists as nested objects:

```
{ "id": "...", "name": "Favorites", "items": [{ "product": { ... } }] }
```

Problem: Merge requires comparing item sets across two lists. With nested objects, this means traversing two entire subtrees, rebuilding the combined structure, and risking data duplication if the same product object appears in multiple places.

Revised approach: Normalized store — wishlists and items as separate flat arrays, linked by IDs:

```
WishlistStore = { wishlists: Wishlist[], items: WishlistItem[] }
```

Result: Merge becomes a pure set operation on `product_id` arrays. Duplicate detection is $O(1)$ per lookup via a `Set`. Cascade delete is a single filter call. This is the same relational pattern used in databases — it made the merge algorithm significantly simpler and more testable.

Merge Semantics — Copy → Destructive

First instinct: Copy source items into target, leave source intact.

Problem: The user now has two lists containing largely the same products. They must manually decide what to delete. "Merge" becomes "Copy with extra steps."

Revised approach: Merge is destructive. Source is deleted after all items are transferred to the target. The user ends with exactly one list — the combined result. See [§ Why Destructive Merge](#) for full reasoning.

State Updates — Sequential → Atomic

First instinct: Call `createWishlist()`, then call `addItemToWishlist()` in sequence when "Create list and add product" is triggered.

Problem: React `setState` is asynchronous. The second call reads a stale store snapshot — the newly created list doesn't exist in it yet. The product silently never gets added.

Revised approach: All compound operations return a new `WishlistStore` computed in a single function call. The context applies it via one `setStore` call, equivalent to a DB transaction. This pattern was then applied consistently across all mutations.

Assumptions

- A single user per browser profile; no multi-user support.
 - Products are static seeded data (20 items, 5 categories). No product admin or API.
 - Wishlist names are not enforced to be unique — uniqueness is by ID.
 - A product can appear in multiple wishlists, but only once within any single wishlist.
 - Merge is one-way and destructive: the source wishlist is deleted after a successful merge.
 - `localStorage` is the only persistence mechanism; data is tied to the origin and browser.
 - Modern browsers only (`crypto.randomUUID`, ES2020+).
-

Architecture

- **Framework:** React 18 + Vite + TypeScript
- **Routing:** React Router v7
- **Styling:** Tailwind CSS 3.4 with shadcn/ui components
- **State:** React Context + custom `useLocalStorage` hook
- **Storage:** `localStorage` under the key `wishlist_store_v1`
- **Icons:** lucide-react
- **Notifications:** sonner toast

Directory Layout

```
src/
├── components/
│   ├── layouts/           # Header, AppLayout
│   ├── products/         # ProductCard, ProductGrid, SearchBar, CategoryFilter
│   └── wishlists/        # WishlistCard, WishlistDetail, AddToWishlistModal,
MergeWizard
├── contexts/             # WishlistContext
├── data/                 # Static product catalog
├── hooks/                # useLocalStorage
├── lib/                  # wishlist utilities (merge algorithm, CRUD, selectors)
├── pages/                # ProductsPage, WishlistsPage, WishlistDetailPage
└── types/                # TypeScript domain types
```

Tradeoffs

LocalStorage vs. a Backend

	LocalStorage	Remote Database
Setup	Zero — native browser API	Requires server, auth, schema
Speed	Instant reads/writes	Network latency

	LocalStorage	Remote Database
Scope	This browser only	Shared across devices and users
Durability	Lost when browser data is cleared	Durable and recoverable
Capacity	~5 MB per origin	Practically unlimited
Chosen because	No backend was specified; acceptable for a single-user prototype	—

React Context vs. Redux / Zustand

	Context API	Redux / Zustand
Boilerplate	Minimal	Moderate (Redux) / low (Zustand)
DevTools	None built-in	Redux DevTools, time-travel
Scale	Good for small apps	Better for large apps or many consumers
Testing	Wrap in a provider	Mock store / actions
Chosen because	Single user, single store, no cross-cutting state. The added indirection of a separate state library is unnecessary here.	—

Set-Union Merge vs. First-Write-Wins / Last-Write-Wins

Strategy	Behavior	Chosen?
Set-union (chosen)	All unique products from both lists appear in the target. Duplicates skipped silently.	✓
First-write-wins	Source items that already exist in target are rejected; merge may partially fail	✗ Confusing for users
Last-write-wins	Source items overwrite target items with matching product_id	✗ Could silently replace data

Set-union was chosen because it is lossless (no data is destroyed beyond removing the source list itself) and predictable (users always end up with a superset).

Design Decisions

Decision	Rationale
Atomic store updates	Every utility function returns a complete new <code>WishlistStore</code> ; the context applies it in one <code>setStore</code> call — equivalent to a transaction. Prevents partial-update bugs.

Decision	Rationale
Normalized data model	Separate flat arrays for wishlists and items, linked by IDs. Makes merge a pure set operation. Avoids nested object duplication.
product_id as duplicate key	Stable identity. Name or price fields can change; ID cannot.
shadcn/ui + Tailwind	Provides accessible, composable components without building a custom component library.
Static product seed	Keeps the project self-contained and fully deployable without a data service.
Versioned storage key (_v1)	Allows a future schema migration to introduce a _v2 key without breaking existing stored data.

Why Destructive Merge

When merging two wishlists, the source list is deleted after its items are transferred. This was a deliberate decision, not an oversight.

The alternative — keep the source — creates a worse problem: the user now has two largely-identical lists. To clean up, they must manually decide which one to delete. The "merge" has created more work, not less.

Merge should mean combine, not copy. A user who merges "Birthday Ideas" into "Gifts" expects to end up with one list called "Gifts." They do not expect to also keep "Birthday Ideas" around.

The risk of data loss is mitigated by:

1. A merge preview step showing exactly how many items transfer and how many are skipped.
2. An explicit confirmation button.
3. A post-merge toast with the counts.
4. The operation being atomic — it either fully completes or does not happen.

Merge Algorithm

Implemented in `src/lib/wishlistUtils.ts` as `mergeWishlists(store, sourceId, targetId)`.

Steps:

1. **Validate** that `source` $\neq$ `target` and both exist. Throw a descriptive error otherwise.
2. **Collect** all `product_ids` from source wishlist items.
3. **Build a Set** of `product_ids` already present in the target ($O(T)$ build, $O(1)$ lookup).
4. **Iterate** source products:
 1. Already in target $\rightarrow$ increment `skippedDuplicates`, skip.
 2. New to target $\rightarrow$ create a fresh `WishlistItem` record, increment `newItemsAdded`.
5. **Rebuild items array:** remove all source items, append new target items.
6. **Rebuild wishlists array:** remove source, update target's `updated_at`.
7. **Return** `{ store, newItemsAdded, skippedDuplicates }` — caller applies atomically.

Complexity:

- Time: $O(S + T)$ — one pass over source, one pass over target to build the dedup set.
- Space: $O(S + T)$ — the dedup Set plus the new items array.

Data Model

Product

```
interface Product {  
  id: string;  
  name: string;  
  description: string;  
  price: number;  
  image_url: string;  
  category: string;  
}
```

Wishlist

```
interface Wishlist {  
  id: string;  
  name: string;  
  created_at: string; // ISO 8601  
  updated_at: string; // ISO 8601  
}
```

WishlistItem

```
interface WishlistItem {  
  id: string;  
  wishlist_id: string;  
  product_id: string;  
  added_at: string; // ISO 8601  
}
```

Persisted Store

```
interface WishlistStore {  
  wishlists: Wishlist[];  
  items: WishlistItem[];  
}
```

Edge Cases

Scenario	Handling
Merge same wishlist	Blocked with an explicit validation error before any state change.
Missing source or target	Blocked with a "wishlist not found" error.
Source empty	Source is deleted; target is unchanged.
Target empty	All source items are copied into the target.
All items duplicated	Target item count does not increase; source is still deleted.
Add duplicate product	Utility returns <code>null</code> ; context shows an error toast; state unchanged.

Scenario	Handling
Delete wishlist	Cascades: all <code>WishlistItem</code> rows for that wishlist are removed in the same state update.
Corrupted <code>LocalStorage</code>	<code>useLocalStorage</code> catches JSON parse errors and falls back to the empty store.
Missing product ID in wishlist	UI renders an "unavailable" placeholder; item can still be removed.
Long wishlist names	Input capped at 80 characters; layout truncates with ellipsis.

Verification Matrix

Test Case	Input	Expected Result	Status
Merge with duplicates	Source: [A, B], Target: [B, C]	Target: [B, C, A]; Source deleted; skipped=1, added=1	✓ Pass
Merge empty source	Source: [], Target: [A]	Target: [A]; Source deleted	✓ Pass
Merge into empty target	Source: [A, B], Target: []	Target: [A, B]; Source deleted	✓ Pass
Merge same list	<code>sourceId === targetId</code>	Error thrown; no state change	✓ Pass
Merge all duplicates	Source: [A], Target: [A]	Target: [A]; Source deleted; skipped=1, added=0	✓ Pass
Add duplicate product	Add P-001 to list that already has P-001	Error toast; item count unchanged	✓ Pass
Delete wishlist	Delete list with 3 items	List removed; all 3 <code>WishlistItems</code> removed	✓ Pass
Browser refresh	Add 2 items, refresh	Both items restored from <code>localStorage</code>	✓ Pass
Corrupted storage	Set <code>wishlist_store_v1</code> to <code>{bad json}</code>	App loads with empty state; no crash	✓ Pass
Rename with blank name	Rename → " " (spaces only)	Rejected; current name preserved	✓ Pass

Future Improvements

- **Functional updater in `useLocalStorage`:** Allow `setStore(prev => compute(prev))` to eliminate stale-closure risk during rapid mutations.

- **useReducer migration:** Consolidate all mutations through a single reducer for easier testing and potential undo/redo support.
 - **Memoized selectors:** Pre-compute lookup maps (`productId` → `wishlistIds`, `wishlistId` → `count`) to eliminate $O(n)$ re-render cascades.
 - **Post-merge redirect:** After merging from a wishlist detail page, navigate automatically to the target wishlist.
 - **Wishlist name uniqueness:** Prevent duplicate display names (case-insensitive check on create/rename).
 - **Image performance:** Add explicit `width/height` attributes and `fetchpriority="high"` to above-the-fold images.
 - **Export / import:** Allow users to back up and restore their wishlist store as a JSON download.
 - **Automated tests:** Add Vitest unit tests for `wishlistUtils.ts` (pure functions, no DOM required) and React Testing Library integration tests for the merge wizard flow.
-

Testing Summary

Method	Coverage
Build verification	<code>npm run build</code> — no TypeScript errors, no missing imports
Linting	<code>npm run lint</code> — passes with zero warnings
Manual QA	Create, rename, delete wishlists; add and remove products; full merge scenarios (see Verification Matrix above); <code>localStorage</code> restore; corrupted storage fallback; 375px mobile layout; keyboard-only navigation

No automated unit or E2E test suite is currently included. The merge algorithm (`wishlistUtils.ts`) consists of pure functions with no side effects, making it the highest-priority target for a future Vitest suite.